LABORATORY FOR
COMPUTER SCIENCE



MASSACHUSETTS
INSTITUTE OF
TECHNOLOGY

MIT/LCS/TM-175

ON TIME VERSUS SPACE III

Albert R. Meyer
Daniel Weise
Michael C. Loui

September 1980

545 TECHNOLOGY SQUARE, CAMBRIDGE, MASSACHUSETTS 02139

On Time Versus Space III

Albert R. Meyer
Daniel Weise
Michael C. Loui

MIT Laboratory for Computer Science
545 Technology Square
Cambridge, Massachusetts 02139

September 2, 1980

Abstract

Paul and Reischuk devised space efficient simulations of logarithmic cost random access machines and multidimensional Turing machines. We simplify their general space reduction technique and extend it to other models of computation, particularly to the class of storage modification machines (SMM), a model of list processing. Every SMM of time complexity $t(n)$ can be simulated by an SMM of space complexity $t(n)/\log t(n)$.

Key Words and Phrases:

Pebble game, time/space tradeoffs, storage modification machines, time complexity, space complexity.

This work was supported in part by the National Science Foundation, Grant No. MCS-77-19754 and in part by the Fannie and John Hertz Foundation.

Introduction

The question of whether space is more powerful than time in a computation was recently answered affirmatively by Hopcroft, Paul, and Valiant [HPV] for multitape Turing machines. Specifically, they showed that every deterministic multitape Turing machine of time complexity $t(n)$ can be simulated by a deterministic Turing machine of tape complexity $t(n)/\log t(n)$. Paul and Reischuk[PR] extended this result to logarithmic cost random access machines and a modified form of this result to multidimensional Turing machines. We extend and generalize Paul and Reischuk's results to other models of computation and simplify their algorithm.

Storage Modification Machines

Our model of computation will be Schönhage's *Storage Modification Machines* (SMM's). The following is a terse (but complete) description of them modified from [SMM], to which the reader is referred for more information. The storage structure for an SMM is graphlike – it consists of *cells* and *pointers*[†]. The pointers are labeled with *directions* from a finite alphabet Δ. Δ is an individual feature of a given machine. *A Δ–structure* consists of cells and pointers and is defined as a finite digraph where the pointers are labeled with the elements of Δ. Each cell has $|\Delta|$ pointers originating from it no two of which have the same label. In addition, there is one distinguished cell, called the *center* of the structure which allows access to the cells of the Δ–structure. The center can be considered as the endpoint of an additional pointer coming from outside. Note that there is no information stored in the cells, only in the the pattern of the pointers in the Δ–structure.

Formally, a Δ–structure is defined as a triple $S = (X, a, p)$, where X denotes the finite set of cells, $a \in X$ is the center of S, and $p = \{p_\delta | \delta \in \Delta\}$ is the family of pointer mappings $p_\delta: X \to X$; $p_\delta(x) = y$ means that the pointer with label δ originating from x goes to y.

In a natural way every word $W \in \Delta^*$ determines a path in the structure S starting at the center and following the labeled pointers according to the sequence of symbols in W. This induces a map $p^*: \Delta^* \to X$ recursively defined by

$$p^*(\Lambda) = a$$
$$p^*(W\delta) = p_\delta(p^*(W)) \text{ for all } \delta \in \Delta, W \in \Delta^*$$

where Λ denotes the empty word.

The finite control for an SMM is given as a program consisting of a sequence of *labelled instructions*. Labels are written as in ALGOL and are referred to in conditional statements which transfer control to other places in the program. The machine starts its work with the first instruction of the program and a given storage structure. There are six kinds of instructions, some of which may alter the current Δ–structure. These instructions are:

$$accept, reject, center, new, assign, if.$$

accept

The machine stops. An SMM *accepts* an initial Δ–structure *iff* the last instruction executed by the SMM was *accept*.

reject

The machine again stops. A *reject* instruction is implicitly performed if the control passes the end of the program.

new

The current storage structure $S = (X, a, p)$ is modified by creating a new cell y; the resulting structure is $(X \bigcup \{y\}, y, \overline{p}_\delta)$ where $\overline{p}_\delta(x) = p_\delta(x)$ for all $x \in X$ and $\overline{p}_\delta(y) = a$ for all $\delta \in \Delta$.

† The technical terms are *vertices* and *arcs*. We use the terms "pointer" and "cell" because they are more connotative of a storage structure for a machine. Later when we need to talk explicitly about graphs *per se* we will use the terms *vertex* and *edge*. "Cell" and "pointer" will be the only terms used to describe Δ–structures.

center δ

This changes the center node. $S = (X, a, p)$ becomes $(X, p^*(\delta), p)$.

assign $\delta := V$

Here $S = (X, a, p)$ is modified into $\overline{S} = (X, a, \overline{p})$ by new assignment of a pointer. $\delta \in \Delta$ and $V \in \Delta^*$ (where $|V| \leq 2$) determine which pointer shall be directed where: $\overline{p}_b{}^*(\Lambda) = p^*(V)$; all other pointers remain unchanged.

if $\alpha = \beta$ *goto* λ (where $\alpha, \beta \in \Delta \bigcup \{\Lambda\}$)

If in the instantaneous storage structure $p^*(\alpha) = p^*(\beta)$, then control branches to the instruction labeled by λ; otherwise there is no change in the flow of control.

Given an initial Δ–structure we define the *running time* of an SMM program on that structure to be the number of instructions performed. We define the *space* used by an SMM program on the initial structure to be $|X|$ at the time the program halts (*i.e.*, the number of cells in the initial structure plus the number of *new* instructions executed).

Note that we can describe SMM programs using high level programming constructs such as procedures, local variables, and recursion. With a little thought the reader should be able to see how these high level descriptions can be implemented using the basic instruction set of SMM's.

Association of Computations with Graphs

We can associate with a computation of an SMM a directed acyclic graph: for each step i ($1 \leq i \leq t(n)$) we associate a vertex i, and there is an edge from vertex i to vertex j if and only if either $j = i + 1$ or the instruction executed at step j uses a cell which has been most recently updated by the instruction executed at step i. The idea is that there an edge into vertex j if the tail of the edge is at a vertex which "created information" used at step j. Such a graph shall be called a *computation graph*. The usefulness of the computation graph is that we can consider information (a cell) to be potentially "stored" at each vertex. If we allow stored information to be discarded at any time and allow instructions to be executed in any order provided that an instruction at step i may be executed only if all the predecessors to vertex i have the information they created stored in them, we may ask: what is the minimum number of vertices which must simultaneously contain information such that we can execute the last instruction of the computation? The space required by the computation will be related to the answer of this question.

Let $G = (V, E)$ be the computation graph corresponding to the computation of an SMM that runs in time $t(n)$ where $V = \{1, 2, \ldots, t(n)\}$ and let I be the maximum indegree of G. (I is independent of the length of computation and depends only on the instructions of the SMM.) Let $V[a, b]$ be the set of vertices a through b and $E[a, b]$ be the edges of the subgraph of G induced by $V[a, b]$. Let $C[a, b, c] = E[a, c] - E[a, b] - E[b, c]$ for $a \leq b \leq c$. (C stands for Crossing edges — an edge is in $C[a, b, c]$ if and only if its tail is in $V[a, b]$ and its head is in $V[b, c]$.) Define $e = e(a, c) = |E[a, c]|$ and $v = v(a, c) = |V[a, c]| = c - a + 1$ and let $s = s(a, c) = |E[1, \lfloor (a + c)/2 \rfloor]|/e$, $r = r(a, c) = |E[\lfloor (a + c)/2 \rfloor, c]|/e$, and $t = t(a, c) = |C[a, \lfloor (a + c)/2 \rfloor, c]|/e$. Later we will make use of the following properties of the distribution of edges in the computation subgraph induced by $V[a, c]$:

$$r + s + t = 1, \tag{0.1}$$

$$\frac{1}{I+1} \leq r, s \leq \frac{I}{I+1}, \tag{0.2}$$

$$v \leq e \leq vI. \tag{0.3}$$

(0.2) is derived as follows: When $c - a + 1$ is odd, s (or r) achieves its lowest value when $|E[a, (c+a)/2]|$ (or $|E[(c+a)/2, c]|$) is as small as possible and $|C[a, (c+a)/2, c]| + |E[(c+a)/2, c]|$ (or $|C[a, (c+a)/2, c]| + |E[a, (c+a)/2]|$) is as large as possible. These minimum and maximum values are $(c-a)/2$ and $I(c-a)/2$. Therefore

$$r, s \geq \frac{\frac{c-a}{2}}{\frac{c-a}{2} + \frac{I(c-a)}{2}} = \frac{1}{I+1}.$$

Reversing the argument we have

$$r, s \leq \frac{\frac{(c-a)I}{2}}{\frac{c-a}{2} + \frac{(c-a)I}{2}} = \frac{I}{I+1}.$$

When $c - a + 1$ is even the ratios defining the maximum and minimum values do not work out as cleanly. However, we can approximate the actual inequalities for r and s by the inequalities above when $c - a + 1$ is even.

Description of the Space Saving Algorithm

We now propose an SMM which simulates encoded SMM's. Let *Smachine* be an SMM with alphabet Δ which we wish to simulate. Δ is encoded as a list of cells with one cell for each element of Δ. *Smachine*'s program is encoded as a list of cells with a one to one correspondence between instructions and cells, where the actual instruction is encoded on a cell by the cell's pointer patterns to other cells in this list and to cells in the "alphabet" list. Each cell created by *Smachine* (such cells are called *virtual cells*) will be represented by cells (called *representation cells*) whose alphabet is $\Delta \bigcup \vartheta \bigcup \varphi$.† ϑ and φ are known collectively as the *address pointer* (*address* for short) and point into a reserved set of cells of cardinality $\lceil \sqrt{t(n)} \rceil$. This yields $t(n)$ unique addresses which, when properly encoded and interpreted, can be considered as strictly ordered; when given an address the *increment* procedure returns the next larger one.

We now describe the procedure *SIMULATE* that simulates SMM programs. *SIMULATE* has three parameters:

m, an integer in the range 1 to $t(n)$ specifying how long to run the simulation,

question, a question to be answered when the simulation ends,

and *oracle*, a subroutine which answers questions about the configuration at which the simulation is to begin.

There are four types of questions which can be asked of *SIMULATE*. These are "What is the address of your center cell," "Which instruction should be executed next," "What is the largest address you have seen," and "What is the address of the cell pointed to by the δ pointer of cell <*address*>?"

The oracle conceptually represents a *state* of a computation. The state comprises a virtual storage structure and a value of an instruction counter. The oracle is asked exactly the same types of questions as *SIMULATE*.

When it is done, *SIMULATE* returns one value – the answer to the question.

Procedure *SIMULATE*(*m*, *question*, *oracle*)

SIMULATE consists of three independent routines that are run in *parallel space*: they are executed in parallel; however, whenever one of the strategies attempts to create an *nth* cell it is temporarily suspended until the other strategies have each tried to create an *nth* cell. When one strategy halts with an answer, the other strategies are stopped, and then *SIMULATE* halts and returns the answer. Consequently, *SIMULATE* requires only three times the space of the strategy that requires the least space.

† ϑ and φ are pronounced *vair-theta* and *vair-phi*. Also, the real alphabet for representation cells may be larger than described here, but the extra pointers are extraneous to this discussion.

Strategy 1:

Strategy 1 begins by querying *oracle* for the value of instruction counter it should start with, the address of *oracle*'s center cell, and the largest address used by the oracle. A representation cell a for the center cell to be used during the simulation is created; the address pointer of a is set to the address of the oracle's center cell and its other pointers are all set to *nil*†. The simulation is then carried out step by step. To simulate a *new* instruction a new representation cell is created. Its Δ pointers are set to point to the current center and its address pointer is set to one greater than the last address assigned (which, for the first address to be assigned, is the largest address used by *oracle*).

Whenever a pointer δ is used which points to *nil* then it is made to point to a cell in a way which reflects the structure defined by *oracle*. That is, if δ originates from the representation cell having address x, then it is made to point to the representation of the cell which is pointed to by the δ pointer of the cell having address x in the *structure defined by the oracle*. This may entail creating a new representation cell for the cell to be pointed to if one does not already exist.

Strategy 2:

$SIMULATE$ recursively invokes $SIMULATE(\lceil m/2 \rceil, question, oracle')$ where *oracle'* is the procedure that on input *question'* invokes and returns the output of $SIMULATE(\lfloor m/2 \rfloor, question', oracle)$. This breaks $SIMULATE$ into two independent halves. Questions generated by the second half are answered by running the first half of the simulation. The space required for this strategy is the sum of the space used by the second half plus the space used by the first half plus any overhead incurred in making the recursive call.

Strategy 3:

$SIMULATE$ recursively invokes $SIMULATE(\lceil m/2 \rceil, question, oracle'')$. *oracle''* keeps track of three items: the number of questions it can safely answer, the number of questions it has been asked so far, and a table of answers to selected questions. The number of questions it can safely handle is originally 0. When asked its nth question where n is less than or equal to the number of questions it can safely answer, it first tries to answer the question using its table of answers to selected questions; if the answer is not there, it asks *oracle* the question.

When asked its nth question, *question'*, where n is greater than the number of questions it can safely handle, it first kills the above recursive call to $SIMULATE$ and garbage collects any cells that call used‡. It then compares the answers given by $oracle(question')$ and $SIMULATE(\lfloor m/2 \rfloor, question', oracle)$; if they differ, it places *question'* and the answer given by the call to $SIMULATE$ in the table of selected answers. It then kills itself and invokes $SIMULATE(\lceil m/2 \rceil, question, oracle'')$ where the number of questions which can be safely handled is increased by 1.

This strategy keeps the two halves of the simulation from being active simultaneously. The space used by this strategy is the overhead plus the maximum of the space used by the first half of simulation and the space used by the second half of the simulation.

Analysis of Space Requirements

Let $Sp(m, state)$ be the largest space (number of cells) required to compute $SIMULATE(m, question, oracle)$ for any question where *state* is the virtual state defined by *oracle*. The space required for Strategy 3 is

$$\max(Sp(\lfloor m/2 \rfloor, state), Sp(\lceil m/2 \rceil, state')) + |table| + 1 \qquad (1)$$

† *nil* is a reserved cell whose sole purpose is to ground "undefined" pointers.

‡ Garbage collection is method of reusing cells. When a cell is garbage collected it can be reallocated by a *new* statement.

where *table* is the state of the table used by *oracle''* when Strategy 3 succeeds, and *state'* is the state of the computation after $\lfloor m/2 \rfloor$ steps. We are allowed to take the maximum between the two halves because of the garbage collection; without it, we would be forced to take the sum of the space used by the two halves. The additional constant 1 accounts for the "stack cell" allocated for the recursive call of $SIMULATE$ and upon which $SIMULATE$'s arguments are encoded.

The space required by strategy 2 is

$$Sp(\lfloor m/2 \rfloor, state) + Sp(\lceil m/2 \rceil, state') + 1 \tag{2}$$

where we again account for the stack cell and *state'* is the state defined by *oracle'* (which is equivalent to the state defined by *oracle''*).

Let $G = (E, V)$ be the computation graph associated with the computation defined by the simulated program. Assume the notation of the section on associating graphs with computations where a and c are chosen such that they "enclose" the sub-computation of the simulated program when it is started in the state defined by *oracle* and runs for m steps. All logarithms are to the base 2.

Theorem: $Sp(m, state) = O(\frac{e}{\log e})$.
Corollary 1: $Sp(m, state) = O(\frac{m}{\log m})$.
Corollary 2: Let $Space(m)$ = maximum of $Sp(m, state)$ for any *state*. Then $Space(m) = O(\frac{m}{\log m})$.

We prove the theorem by induction on m, the number of steps for which the simulation runs. Let $Q = 2^{I+1}$. Assume, without loss of generality, that for $m \leq Q$ strategy 1 always takes the least space. Clearly, for $m \leq Q$, there exists a k such that

$$Sp(m, state) \leq \frac{ke}{\log e}.$$

Assume, without loss of generality, that $k \geq 10I \log(I+1)$. Let r, s, and t be as in (0.1) through (0.3). Using $Sp(m, state) \leq ke/\log e$ as an induction hypothesis we have

$$Sp(\lfloor m/2 \rfloor, state) \leq \frac{kse}{\log se} \text{ and } Sp(\lceil m/2 \rceil, state') \leq \frac{kre}{\log re}. \tag{3}$$

Using (3) we can bound (2) by

$$\frac{kse}{\log se} + \frac{kre}{\log re} + 1. \tag{4}$$

When strategy 3 halts, the table *oracle''* keeps corresponds to the set $\{v_i | (v_i, v_j) \in C[a, b, c]\}$. Therefore $|table| \leq te$. Using (3) and $r, s \leq I/(I+1)$ from (0.2), we can bound (1) by

$$\frac{k\frac{I}{I+1}e}{\log \frac{I}{I+1}e} + te + 1. \tag{5}$$

In what follows assume $m > Q$, hence $e \geq Q$. Let ϵ denote $\log e/ke$.
Claim 1: If $t \geq (\log(I+1)/\log e) + \epsilon$ then $(4) \leq \frac{ke}{\log e}$.
Claim 2: If $t < (\log(I+1)/\log e) + \epsilon$ then

$$(5) < \frac{ke\frac{I}{I+1}}{\log \frac{I}{I+1}e} + e\left(\frac{\log I + 1}{\log e} + \frac{\log e}{ke}\right) + 1 \leq \frac{ke}{\log e}. \tag{6}$$

Proof of Claim 1:

$$\begin{aligned}
\frac{kse}{\log se} + \frac{kre}{\log re} + 1 &\le \frac{k(s+r)e}{\log \frac{e}{I+1}} + 1 \text{ By (0.2)} \\
&= \frac{k(s+r)e}{\log e - \log(I+1)} + 1 \\
&= k(s+r)e\left(\frac{1}{\log e} + \frac{\log(I+1)}{\log e \log \frac{e}{I+1}}\right) + 1 \text{ using } \tfrac{1}{x-c} = \tfrac{1}{x} + \tfrac{c}{x(x-c)} \\
&= \frac{ke}{\log e}(1-t)\left(1 + \frac{\log(I+1)}{\log \frac{e}{I+1}}\right) + 1 \text{ substituting } 1-t \text{ for } s+r \\
&\le \frac{ke}{\log e}\left(1 - \frac{\log(I+1)}{\log e} - \epsilon\right)\left(1 + \frac{\log(I+1)}{\log \frac{e}{I+1}}\right) + 1 \text{ by hypothesis} \\
&= \frac{ke}{\log e}\left(1 - \frac{\log(I+1)}{\log e}\right)\left(1 + \frac{\log(I+1)}{\log \frac{e}{I+1}}\right) + \frac{ke}{\log e}(-\epsilon)\left(1 + \frac{\log(I+1)}{\log \frac{e}{I+1}}\right) + 1 \\
&= \frac{ke}{\log e} - \left(1 + \frac{\log(I+1)}{\log \frac{e}{I+1}}\right) + 1 \\
&= \frac{ke}{\log e} - \frac{\log(I+1)}{\log \frac{e}{I+1}} \\
&\le \frac{ke}{\log e} \quad \text{since } e \ge 2^{I+1}
\end{aligned}$$

thereby establishing claim 1.

Proof of Claim 2: Multiply through (6) by $\log e/ke$:

$$\frac{I \log e}{(I+1)(\log e + \log(\frac{I}{I+1}))} + \frac{\log(I+1)}{k} + \frac{\log^2 e}{k^2 e} + \frac{\log e}{ke} \le 1. \tag{8}$$

Because $\log e/(\log e + \log(I/(I+1)))$ decreases as e increases, we can replace e by its lower bound of 2^{I+1} in the first term of (8):

$$\frac{I(I+1)}{(I+1)(I+1+\log(\frac{I}{I+1}))} + \frac{\log(I+1)}{k} + \frac{\log^2 e}{k^2 e} + \frac{\log e}{ke} \le 1. \tag{9}$$

That is, (9) implies (8). Because $\log(I/(I+1) < -1.5/I$ (since $\log(I/(I+1)) = -\log((I+1)/I) = -\log(1+1/I) = -(\ln 2)^{-1}(1/I - 1/I + 1^2 + 1/3I^3 - \cdots) > -1.4427(1/I) > -1.5/I$ which implies $\log(I/I+1) < -1.5/I$),

$$\frac{I}{(I+1-\frac{1.5}{I})} + \frac{\log(I+1)}{k} + \frac{\log^2 e}{k^2 e} + \frac{\log e}{ke} \le 1, \tag{10}$$

implies (9). Inequality (10) is implied by

$$\frac{I}{(I+1-\frac{1.5}{I})} + \frac{1}{10I} + \frac{1}{100I} + \frac{1}{10I} \le 1 \tag{11}$$

since we have assumed $k \geq 10I \log(I+1)$. Inequality (11) is equivalent to

$$\frac{200I^3 + 42I^2 + 42I - 63}{200I^3 + 200I^2 - 300I} \leq 1,$$

whose truth the reader can verify for himself for $I \geq 2$. •

We therefore have that for $m > Q$ either strategy 2 or strategy 3 will require $O(e/\log e)$ space, and therefore $Sp(m, state) = O(e/\log e)$ *Q.E.D.*

Proof of Corollary 1: Since $e \leq Im$ and $\log e \geq \log m$ we have that $Sp(m, state) = O(\frac{m}{\log m})$ •
The proof of corollary 2 immediately follows. Therefore, an SMM of time complexity $t(n)$ can be simulated by an SMM of space complexity $t(n)/\log t(n)$.

Relation to other models of computation

These results easily generalize to other models of computation. The space efficiency of the procedure is bounded by the efficiency with which one can store *table* of Strategy 3. For any model of computation where the storage to encode *table* follows the rule that $SPE(table) = |table| f(|table|)$ where $SPE(table)$ is the space to encode *table* and $f(x)$ is positive and non-decreasing, then

$$Sp(m) = O(SPE(e)/\log e) \tag{12}$$

The above conditions on SPE yield that

$$SPE(re) + SPE(se) = (re)f(re) + (se)f(se) \leq (r+s)ef(e) = (r+s)SPE(e), \tag{13}$$

which implies $SPE(re) + SPE(se) \leq (r+s)SPE(e)$. Using (13), (12) can be proved in exactly the same way as the main theorem was proved. For any model of computation, there will always exist a k such that for $m \leq Q$, $Sp(m, state) \leq (kSPE(e)/\log e)$. We then use $Sp(m, state) \leq (kSPE(e)/\log e)$ as the induction hypothesis. Claim 1 and Claim 2 are changed as follows.

Claim 1′: If $t \geq \log(I+1)/loge + \epsilon$ then

$$\frac{kSPE(se)}{\log se} + \frac{kSPE(re)}{\log re} + overhead \leq \frac{kSPE(e)}{\log e}.$$

Claim 2′: If $t < \log(I+1)/loge + \epsilon$ then

$$\frac{kSPE(e\frac{I}{I+1})}{\log \frac{I}{I+1} e} + e\left(\frac{\log(I+1)}{\log e} + \epsilon\right) + overhead \leq \frac{kSPE(e)}{\log e}.$$

In these altered claims ϵ equals *overhead* $\log e/(k)SPE(e)$. The proof of Claim 1′ follows exactly the proof of Claim 1 (the justification for the first step of the proof becomes (13) and (0.2) together, the rest of the proof is algebra). Likewise, the proof of Claim 2′ resembles the proof of claim 2 except that the lower bound for k is adjusted for differing amounts of overhead.

Acknowledgments

We would like to thank Joseph Y. Halpern for many enlightening discussions and for first showing that splitting the computation into two equal halves would yield the intended result.

References

[HPV] HOPCROFT, J., ULLMAN, J., AND VALIANT, L. On time versus space. *J. ACM*, 24. 332-337, 1977

[PR] PAUL, W. J., AND REISCHUK, R. On time versus space II. Proc. of the 20th Annual Symp. on Foundations of Computer Science, 298-306, 1979

[SMM] SCHÖNHAGE, A. Storage Modification Machines. Unpublished manuscript, *Mathematisches Institut, Universitat Tubingen, Germany* 1979